Super Learners

Big Data y Machine Learning para Economía Aplicada

Ignacio Sarmiento-Barbieri

Universidad de los Andes

Agenda

1. Superlearners
2. Ejemplo Algebraico
3. Propiedades Teóricas
4. CV SuperLearner

Agenda

1 Superlearners

2 Ejemplo Algebraico

3 Propiedades Teóricas

4 CV SuperLearner

Recap

- Queremos predecir y en funcion de observables ($\mathbf{X}$)

$$y = f(\mathbf{X}) + u \tag{1}$$

- donde la estimación de f implica la que mejor generalice (prediga mejor fuera de muestra):

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f(\mathbf{X}; \Theta)) \right\} \tag{2}$$

Superlearners: Motivation

- Superlearning is a technique for prediction that involves combining many individual statistical algorithms to create a new, single prediction algorithm that is expected to perform at least as well as any of the individual algorithms.
- The innovation?

Superlearners: Motivation

- Superlearning is a technique for prediction that involves combining many individual statistical algorithms to create a new, single prediction algorithm that is expected to perform at least as well as any of the individual algorithms.
- The innovation?
  - The superlearner uses **cross-validation** to determine how to combine, or *weight*, the individual algorithms based on their out-of-sample predictive performance.
  - The weights are **data-driven**: algorithms that predict better out-of-sample receive higher weights.
  - Formally, the SL is guaranteed to perform asymptotically at least as well as any single algorithm (or any convex combination of algorithms) in the library: the **Oracle Property**.

Superlearners: Algorithm

Denote the library $\mathcal{L}$ and its cardinality as $V(n)$.

1. Fit each algorithm $v \in \mathcal{L}$ on the **entire** data set $\{(y_i, X_i) : i = 1, \ldots, n\}$ to obtain the full-data estimators $\hat{f}_v$, for $v = 1, \ldots, V(n)$.
   - These full-data fits are saved and used in the *final* step to generate predictions.

Superlearners: Algorithm

2. Split the data set into training and validation samples, according to a K-fold cross-validation scheme:
   - Partition $\{1, \ldots, n\}$ into K equal-size groups (folds): $T_1, T_2, \ldots, T_K$.
   - For fold k: the **validation set** is $Te(k) = T_k$ and the **training set** is $T(k) = \{1, \ldots, n\} \setminus T_k$.
   - Let $k(i)$ denote the fold to which observation i is assigned.

Superlearners: Algorithm

3. For each fold k and each algorithm $v \in \mathcal{L}$:
   - Fit algorithm v on the training data $T(k)$.
   - Generate out-of-sample predictions $\hat{f}_v^{(-k)}(X_i)$ for all $i \in Te(k)$.
   - The key here is that the model never sees observation i during training.

Superlearners: Algorithm

4. Bind the out-of-sample predictions from all algorithms and all folds into an $n \times V(n)$ matrix Z:

$$Z = \begin{pmatrix} \hat{f}_1^{(-k(1))}(X_1) & \hat{f}_2^{(-k(1))}(X_1) & \cdots & \hat{f}_V^{(-k(1))}(X_1) \\ \hat{f}_1^{(-k(2))}(X_2) & \hat{f}_2^{(-k(2))}(X_2) & \cdots & \hat{f}_V^{(-k(2))}(X_2) \\ \vdots & \vdots & \ddots & \vdots \\ \hat{f}_1^{(-k(n))}(X_n) & \hat{f}_2^{(-k(n))}(X_n) & \cdots & \hat{f}_V^{(-k(n))}(X_n) \end{pmatrix} \tag{3}$$

- Each entry Z_{iv} is the prediction for observation i by algorithm v, trained on data *that excludes* i.
- Z is the key object: it summarizes out-of-sample performance of all algorithms simultaneously.

Superlearners: Algorithm

5 Propose a family of weighted combinations of the candidate estimators indexed by $\alpha = (\alpha_1, \ldots, \alpha_V)$:

$$\hat{f}_{SL}(x; \alpha) = \sum_{v=1}^{V} \alpha_v \hat{f}_v(x) \tag{4}$$

where the weights are constrained to the **simplex**:

$$\mathcal{A} = \left\{ \alpha \in \mathbb{R}^V \ : \ \alpha_v \geq 0 \ \ \forall v, \quad \sum_{v=1}^{V} \alpha_v = 1 \right\} \tag{5}$$

Superlearners: Algorithm

6 Determine $\hat{\alpha}$ that minimizes the cross-validated risk over all allowed $\alpha \in \mathcal{A}$:

$$\hat{\alpha} = \underset{\alpha \in \mathcal{A}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} L\left(y_i, \sum_{v=1}^{V} \alpha_v \hat{f}_v^{(-k(i))}(X_i) \right) \tag{6}$$

For squared loss $L(y, \hat{y}) = (y - \hat{y})^2$, this becomes:

$$\hat{\alpha} = \underset{\alpha \in \mathcal{A}}{\operatorname{argmin}} \|Z\alpha - y\|^2 \tag{7}$$

$\Rightarrow$ **Non-Negative Least Squares (NNLS)**

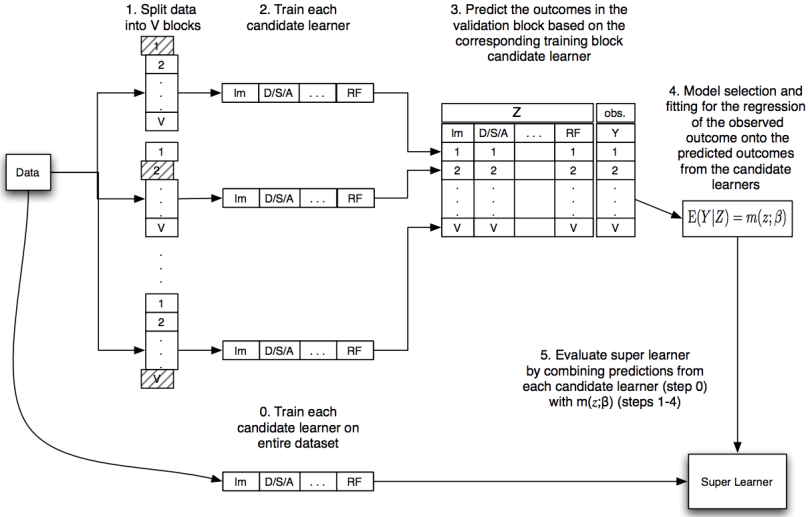
Superlearners: Algorithm

7 Using $\hat{\alpha}$ from Step 6 and the **full-data fits** $\hat{f}_v$ from Step 1, construct the final Super Learner predictor:

$$\hat{f}_{SL}(x) = \sum_{v=1}^{V} \hat{\alpha}_v \hat{f}_v(x) \tag{8}$$

- The weights $\hat{\alpha}_v$ tell us how much each algorithm contributed to predictive accuracy.
- Algorithms with low cross-validated error receive high weight; poor algorithms receive weight ≈ 0.
- In the limit, the SL can *select* the best algorithm ($\hat{\alpha}_v = 1$ for one v) or *combine* several.

Superlearners: Algorithm



Superlearners: Algorithm

Some considerations:

- The super learner theory does not place any restrictions on the family of weighted combinations used for ensembling the algorithms in the library.
- The restriction of the parameter space for α to be the convex combination of the algorithms in the library (the simplex $\mathcal{A}$) provides greater stability of the final super learner prediction.
- The library $\mathcal{L}$ can include any algorithm: OLS, LASSO, ridge, random forests, boosting, neural networks. There are no restrictions!
- A larger library is generally better: more candidates give the SL more to work with.

Agenda

1 Superlearners

2 Ejemplo Algebraico

3 Propiedades Teóricas

4 CV SuperLearner

Ejemplo Algebraico: Setup

Setup:

- $n = 6$ observaciones, $x \in \mathbb{R}$
- $K = 2$ folds
- Librería: $\mathcal{L} = \{\hat{f}_1, \hat{f}_2\}$
  - $\hat{f}_1$: OLS (regresión lineal)
  - $\hat{f}_2$: Árbol de regresión

i	x_i	y_i
1	1	1
2	2	3
3	3	5
4	4	7
5	5	9
6	6	11

Asignación de folds:

- Fold 1: entrenamiento $= \{4, 5, 6\}$, validación $= \{1, 2, 3\}$
- Fold 2: entrenamiento $= \{1, 2, 3\}$, validación $= \{4, 5, 6\}$

Ejemplo Algebraico: Predicciones Cruzadas

OLS ajustado en cada fold de entrenamiento:

- Fold 1 (entrena en $\{(4,7),(5,9),(6,11)\}$): $\hat{f}_1 = 2x - 1 \quad \Rightarrow \quad$ predice $(1,\ 3,\ 5)$ sobre $\{1,2,3\}$
- Fold 2 (entrena en $\{(1,1),(2,3),(3,5)\}$): $\hat{f}_1 = 2x - 1 \quad \Rightarrow \quad$ predice $(7,\ 9,\ 11)$ sobre $\{4,5,6\}$

Árbol (promedio del conjunto de entrenamiento):

- Fold 1 (entrena en $\{7,9,11\}$): $\bar{y} = 9 \quad \Rightarrow \quad$ predice $(9,\ 9,\ 9)$ sobre $\{1,2,3\}$
- Fold 2 (entrena en $\{1,3,5\}$): $\bar{y} = 3 \quad \Rightarrow \quad$ predice $(3,\ 3,\ 3)$ sobre $\{4,5,6\}$

Ejemplo Algebraico: La Matriz Z

Uniendo todas las predicciones fuera de muestra:

$$Z = \begin{pmatrix} \hat{f}_1^{(-1)}(x_1) & \hat{f}_2^{(-1)}(x_1) \\ \hat{f}_1^{(-1)}(x_2) & \hat{f}_2^{(-1)}(x_2) \\ \hat{f}_1^{(-1)}(x_3) & \hat{f}_2^{(-1)}(x_3) \\ \hat{f}_1^{(-2)}(x_4) & \hat{f}_2^{(-2)}(x_4) \\ \hat{f}_1^{(-2)}(x_5) & \hat{f}_2^{(-2)}(x_5) \\ \hat{f}_1^{(-2)}(x_6) & \hat{f}_2^{(-2)}(x_6) \end{pmatrix} = \begin{pmatrix} 1 & 9 \\ 3 & 9 \\ 5 & 9 \\ 7 & 3 \\ 9 & 3 \\ 11 & 3 \end{pmatrix} \tag{9}$$

- Cada columna: predicciones cruzadas de un algoritmo.
- Cada fila: predicción de cada algoritmo para esa observación, **sin haberla visto en entrenamiento**.

Ejemplo Algebraico: Optimización de Pesos

Paso 6: Resolver el NNLS sobre la matriz Z:

$$\hat{\alpha} = \underset{\alpha \in \mathcal{A}}{\operatorname{argmin}} \|Z\alpha - y\|^2 = \underset{\substack{\alpha_1, \alpha_2 \geq 0 \\ \alpha_1 + \alpha_2 = 1}}{\operatorname{argmin}} \left\| \begin{pmatrix} 1 & 9 \\ 3 & 9 \\ 5 & 9 \\ 7 & 3 \\ 9 & 3 \\ 11 & 3 \end{pmatrix} \begin{pmatrix} \alpha_1 \\ \alpha_2 \end{pmatrix} - \begin{pmatrix} 1 \\ 3 \\ 5 \\ 7 \\ 9 \\ 11 \end{pmatrix} \right\|^2 \tag{10}$$

Resolviendo tenemos que:

$$\hat{\alpha}_1 = 1, \qquad \hat{\alpha}_2 = 0 \tag{11}$$

OLS predice perfectamente (lleva todo el peso)

Ejemplo Algebraico: Predictor Final

Paso 7: Con los pesos $\hat{\alpha}$ y los ajustes sobre datos completos $\hat{f}_v^{full}$:

$$\hat{f}_{SL}(x) = \hat{\alpha}_1 \cdot \hat{f}_1^{full}(x) + \hat{\alpha}_2 \cdot \hat{f}_2^{full}(x) = 1 \cdot \hat{f}_1^{full}(x) + 0 \cdot \hat{f}_2^{full}(x) = \hat{f}_1^{full}(x) \tag{12}$$

- En este caso, el Super Learner **selecciona** al OLS, asignándole todo el peso, porque los datos son lineales y el OLS los captura perfectamente.
- Con datos **no lineales** (como en el cuaderno) ambos algoritmos reciben pesos positivos y el SL **combina** sus fortalezas. Esa es la intuición detrás del ensemble.

Agenda

1 Superlearners

2 Ejemplo Algebraico

3 Propiedades Teóricas

4 CV SuperLearner

El Teorema Oracle

- ¿Por qué funciona el Super Learner?

- Oracle Inequality (van der Laan, Polley & Hubbard, 2007)

  Sea $\hat{f}^*_{oracle} = \sum_{v=1}^{V} \alpha^*_v \hat{f}_v$ *la combinación convexa óptima si pudiéramos observar el verdadero riesgo (el "oracle"). Entonces, bajo condiciones de regularidad:*

$$E\left[R(\hat{f}_{SL})\right] \leq E\left[R(\hat{f}^*_{oracle})\right] + O\left(\frac{\log V(n)}{n}\right) \tag{13}$$

  donde $R(f) = E[L(Y, f(X))]$ *es el riesgo verdadero.*

El Teorema Oracle: Implicaciones

1. **No hay riesgo en agrandar la librería.**
   Agregar algoritmos aládecuadamente no puede perjudicar al SL. En el peor caso, un algoritmo malo recibe peso 0.

2. **No se necesita saber cuál algoritmo es el mejor.**
   El SL lo determina automáticamente via la validación cruzada. No hay "selección de modelos" manual.

3. **No se hacen supuestos sobre la forma de f.**
   El SL es un estimador no paramétrico: no asume linealidad ni ninguna estructura particular en los datos.

4. **El SL es *adaptive*:** si los datos son lineales, se comporta como OLS; si son no lineales, combina o selecciona el mejor algoritmo no lineal.

Agenda

1 Superlearners

2 Ejemplo Algebraico

3 Propiedades Teóricas

4 CV SuperLearner

CV SuperLearner: El Problema

- Podemos usar el riesgo estimado del SL como medida de error generalizacion?
  - Al minimizar $\hat{R}_{CV}$ sobre $\alpha \in \mathcal{A}$, el SL selecciona el $\hat{\alpha}$ que "luce mejor" en esa CV particular.
  - Esto introduce un **sesgo**: el riesgo interno tiende a subestimar el error de generalización real.
  - El mismo problema surge con cualquier selector de modelos: reportar el CV error del modelo ganador es optimista.

CV SuperLearner: La Solución

La solución es envolver al Super Learner completo en una capa adicional de cross-validation:

1. Dividir los datos en K^{outer} folds externos (ej. $K^{outer} = 10$).
2. Para cada fold externo k:
   - Entrenar un **Super Learner completo**, incluyendo su CV interna para elegir $\hat{\alpha}$, sobre los datos de entrenamiento externo $\mathcal{T}(k)$.
   - Generar predicciones sobre el fold de validación externo $\mathcal{T}_e(k)$: $\hat{f}_{SL}^{(-k)}(x_i)$ para todo $i \in \mathcal{T}_e(k)$.
3. Calcular el **riesgo CV-SL**: estimador del error de generalización:

$$\hat{R}_{CV\text{-}SL} = \frac{1}{n}\sum_{i=1}^{n} L\left(y_i, \hat{f}_{SL}^{(-\kappa(i))}(x_i)\right) \tag{14}$$

donde $\kappa(i)$ es el fold externo de la observación i.

Implementación: SuperLearner

photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/